



南开大学
Nankai University

诗文少年项目说明文档

项 目 名 称： 诗文少年

学 校： 南开大学

团 队 成 员： 乔昊 刘蓓蓓

刘可新 王前

指 导 老 师： 孙永谦

2025 年 6 月 28 日

目录

1 项目概述	4
1 需求分析	4
1.1 市场背景	4
1.2 用户需求细分	4
2 核心使命	4
3 项目目标与意义	5
3.1 具体目标	5
4 技术亮点	5
2 项目功能模块	6
1 用户认证模块	6
1.1 登录	6
1.2 注册	8
1.3 注销	9
2 诗词创作模块	10
2.1 个性化诗词内容提交	10
2.2 诗词润色	12
2.3 诗词评分	15
2.4 诗词推荐	17
3 文言文分析模块	19
3.1 逐句翻译	19
3.2 重点字词解释	20
3.3 重点字词解释	20
3.4 主旨分析	22
4 素材库模块	23
5 图文生成模块	25
3 项目架构	28
1 前端架构	28
1.1 技术选型	28
1.2 模块化设计	28

1.3 核心交互流程	28
2 后端架构	29
2.1 技术栈	29
2.2 模块化设计	29
2.3 API 网关设计	29
3 系统部署架构	29
3.1 开发环境	29
3.2 第三方 API 调用	30
4 数据库操作技术	31
1 Python 与 SQLite 交互	31
2 数据库安全措施	31
3 数据库备份与恢复	32
5 API 接口说明	33
1 API 设计规范	33
1.1 URL 规范	33
1.2 响应格式	33
2 用户认证模块	33
2.1 用户登录	33
2.2 用户注册	34
3 诗词创作模块	34
3.1 诗词提交	34
3.2 诗词润色	35
3.3 诗词评分	35
4 文言文分析模块	35
4.1 文言文翻译界面	35
4.2 重点字词解释	36
5 素材库模块	36
5.1 获取素材列表	36
5.2 获取单个素材详情	36
6 图文生成模块	36
6.1 根据图片生成诗词	36
6 部署与运行	38
1 环境准备	38

- 2 配置文件 38
- 3 运行步骤 38
- 7 差异化创新 39
 - 1 产品定位创新 39
 - 1.1 多场景覆盖 39
 - 1.2 精准用户画像 39
 - 2 技术应用创新 39
 - 2.1 AI 多模态融合 39
 - 2.2 自适应学习系统 39
 - 3 用户体验创新 40
 - 3.1 低门槛创作 40
 - 3.2 沉浸式学习环境 40

项目概述

1 需求分析

1.1 市场背景

随着国家对传统文化的重视，古诗文学习需求日益增长，但传统教学方式存在以下痛点：

- 中学生缺乏创作指导工具，难以掌握诗词格律与意境表达
- 文言文解析依赖教师经验，缺乏标准化分析工具
- 诗词学习素材分散，缺乏系统性资源整合
- 文化传播方式单一，难以吸引年轻群体

1.2 用户需求细分

1. 诗词爱好者：希望获得诗词创作灵感、提高创作水平、学习文言文知识
2. 学生群体：需要辅助学习古诗文、提高语文作文水平，尤其针对中考高考文言文阅读需求
3. 教育工作者：寻找数字化教学资源、辅助教学活动，提升课堂互动性
4. 文化研究者：获取诗词分析工具、研究诗词文化，挖掘传统文化现代价值

2 核心使命

“诗文少年”是一个基于 AI 技术的诗词写作助手平台，旨在借助火山引擎与 ChatAny-where 等先进 AI 能力，为中学生及诗词爱好者提供一站式创作辅助服务，具体包括：

- 创作赋能：通过 AI 润色与评分系统，帮助用户提升诗词写作水平
- 文化传承：以数字化方式传承和弘扬中华优秀传统文化古诗文，降低学习门槛
- 教育创新：为语文教育提供智能化工具，丰富教学手段
- 技术融合：探索大语言模型在传统文化领域的应用场景

3 项目目标与意义

3.1 具体目标

1. 开发集诗文创作、分析、素材检索于一体的多功能平台
2. 实现准确率超过 90% 的文言文逐句翻译与语法分析
3. 建立包含 10,000+ 经典诗词的素材库，支持智能检索
4. 提供友好的移动端体验，覆盖 80% 以上的中学生用户群体
5. 在项目发布后 6 个月内积累 10,000+ 活跃用户

4 技术亮点

- 多模态交互：支持文本输入与图片上传，实现图文互译与生成
- 精准分析：基于火山引擎大模型，实现文言文语法解构与意境分析
- 个性化推荐：根据用户创作历史，智能推荐适合的学习素材与创作建议
- 轻量化部署：采用 Flask 微框架与 SQLite 数据库，降低服务器成本

项目功能模块

1 用户认证模块

1.1 登录



The image shows a user login interface. At the top left, there is a link labeled "← 返回主页". The main heading is "用户登录" (User Login) with a person icon. Below the heading is a welcome message: "欢迎使用AI写作助手，请登录体验更多功能". There are two input fields: "用户名" (Username) with the placeholder "请输入用户名" and "密码" (Password) with the placeholder "请输入密码". A large "登录" (Login) button is below the password field. At the bottom, there are links for "注册账号" (Register Account) and "忘记密码?" (Forgot Password?).

← 返回主页

用户登录

欢迎使用AI写作助手，请登录体验更多功能

用户名

请输入用户名

密码

请输入密码

登录

注册账号 | 忘记密码?

图 2.1: 登录界面

- 功能描述：用户输入用户名和密码，系统验证用户信息，验证通过则将用户信息存入会话并跳转首页。
- 技术实现：通过 /login 接口查询数据库，使用 check_password_hash 验证密码。以下是 Flask 代码：

```

1 from flask import Flask, request, session
2 from werkzeug.security import check_password_hash
3 import sqlite3
4
5 app = Flask(__name__)
6 app.secret_key = 'your_secret_key'
7
8 @app.route('/login', methods=['POST'])
9 def login():
10     username = request.form.get('username')
11     password = request.form.get('password')
12     conn = sqlite3.connect('users.db')
13     cursor = conn.cursor()
14     cursor.execute('SELECT password FROM users WHERE username =?', (username,))
15     result = cursor.fetchone()
16     if result and check_password_hash(result[0], password):
17         session['username'] = username
18         return '登录成功'
19     return '登录失败'
20
21 if __name__ == '__main__':
22     app.run()

```

1.2 注册



The image shows a user registration interface. At the top left, there is a link labeled "← 返回主页". The main title is "用户注册" (User Registration) with a user icon. Below the title is a subtitle "欢迎注册AI写作助手账号" (Welcome to register AI writing assistant account). The form consists of three input fields: "用户名" (Username) with placeholder "请输入用户名", "密码" (Password) with placeholder "请输入密码", and "确认密码" (Confirm Password) with placeholder "请再次输入密码". At the bottom of the form is a large "注册" (Register) button with a user icon. Below the button is a link "已有账号? 登录" (Already have an account? Login).

图 2.2: 注册界面

- 功能描述：用户输入用户名、密码和确认密码，系统验证信息有效性。
- 技术实现：通过 `/register` 接口验证输入，使用 `generate_password_hash` 加密密码。以下是注册代码：

```

1 from flask import Flask, request
2 from werkzeug.security import generate_password_hash
3 import sqlite3
4
5 app = Flask(__name__)
6
7 @app.route('/register', methods=['POST'])

```

```

8  def register():
9      username = request.form.get('username')
10     password = request.form.get('password')
11     confirm_password = request.form.get('confirm_password')
12     if password != confirm_password:
13         return '两次密码输入不一致'
14     hashed_password = generate_password_hash(password)
15     conn = sqlite3.connect('users.db')
16     cursor = conn.cursor()
17     try:
18         cursor.execute('INSERT INTO users (username, password) VALUES (?,?)', (username, ...
19                             hashed_password))
20         conn.commit()
21         return '注册成功'
22     except sqlite3.IntegrityError:
23         return '用户名已存在'
24     finally:
25         conn.close()
26 if __name__ == '__main__':
27     app.run()

```

1.3 注销

- 功能描述：用户点击登出按钮，系统清除会话信息。
- 技术实现：通过 /logout 接口清除会话。示例代码如下：

```

1  from flask import Flask, session
2
3  app = Flask(__name__)
4  app.secret_key = 'your_secret_key'
5
6  @app.route('/logout')
7  def logout():
8      session.pop('username', None)
9      return '注销成功'
10
11 if __name__ == '__main__':
12     app.run()

```

2 诗词创作模块

2.1 个性化诗词内容提交



输入诗词作品

诗词题目

例如：春夜喜雨

诗词内容

例如：好雨知时节，当春乃发生。随风潜入夜，润物细无声。

诗词风格 *

-- 请选择诗词风格 --

AI将按此风格润色

主题意境 *

例如：思乡、咏物、抒情（不超过20字）

指导AI创作方向，不超过20字

提交诗词

图 2.3: 个性化诗词内容提交界面

- 功能描述：用户输入标题、内容、风格和主题，系统验证并保存为 JSON 文件。
- 技术实现：通过 `/api/submit-poem` 接口接收 JSON 数据。示例代码：

```
1 from flask import Flask, request
2 import json
3
4 app = Flask(__name__)
```

```
5
6 @app.route('/api/submit-poem', methods=['POST'])
7 def submit_poem():
8     data = request.get_json()
9     title = data.get('title')
10    content = data.get('content')
11    style = data.get('style')
12    theme = data.get('theme')
13    poem_data = {
14        'title': title,
15        'content': content,
16        'style': style,
17        'theme': theme
18    }
19    with open('poem.json', 'w', encoding='utf-8') as f:
20        json.dump(poem_data, f, ensure_ascii=False, indent=4)
21    return '诗词提交成功'
22
23 if __name__ == '__main__':
24    app.run()
```

2.2 诗词润色

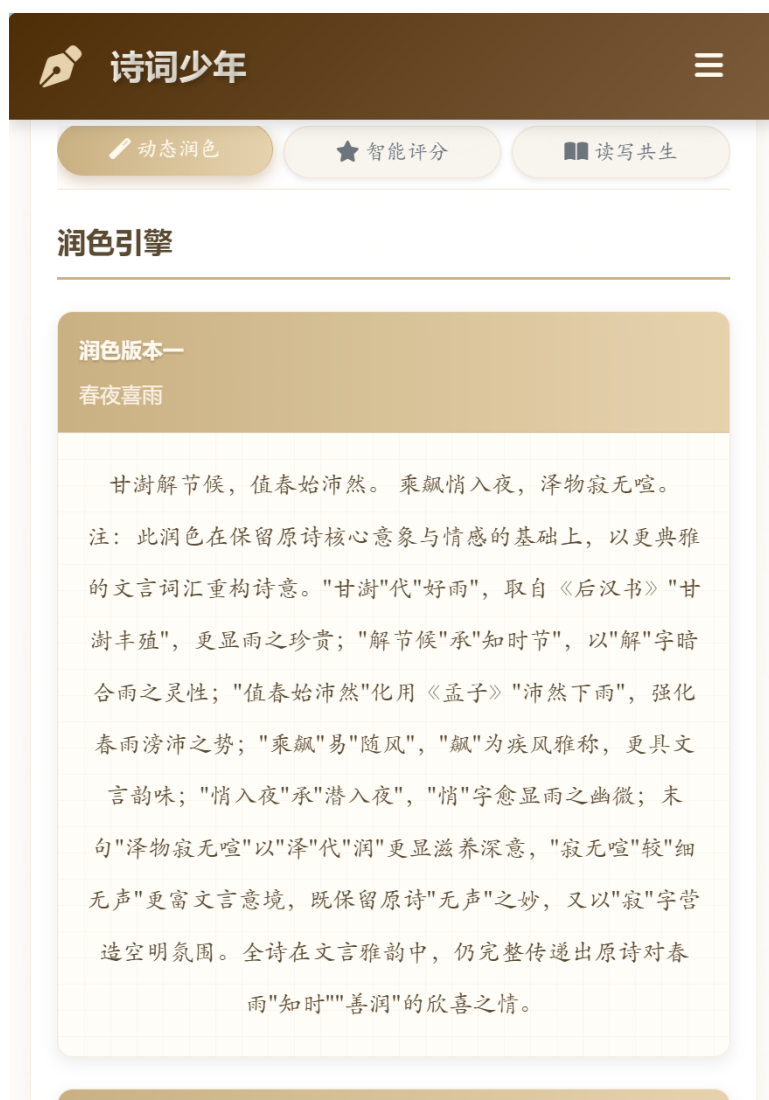


图 2.4: 诗词润色界面 1

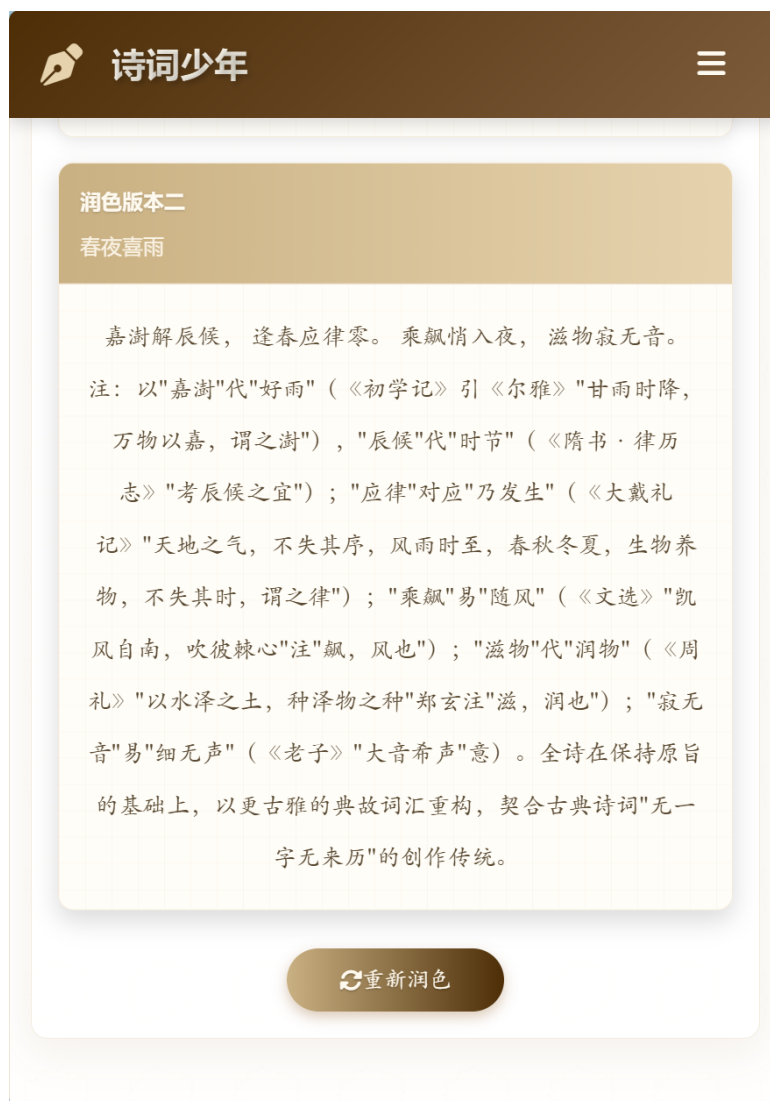


图 2.5: 诗词润色界面 2

- 功能描述：调用火山引擎 API 提供”优雅文言文”和”古典传统”两种风格润色。
- 技术实现：通过 /api/polish-poem 接口调用 polish_with_volcano 函数。以下是简化的示例代码：

```

1 import requests
2
3 def polish_with_volcano(title, content):
4     # 这里需要替换为火山引擎 API 的实际调用代码
5     url = 'https://your-volcano-api-url'
6     data = {
7         'title': title,
8         'content': content
9     }

```

```

10     response = requests.post(url, json=data)
11     return response.json()
12
13 @app.route('/api/polish-poem', methods=['POST'])
14 def polish_poem():
15     data = request.get_json()
16     title = data.get('title')
17     content = data.get('content')
18     result = polish_with_volcano(title, content)
19     return result
20
21 if __name__ == '__main__':
22     app.run()

```

2.3 诗词评分



图 2.6: 诗词评分界面

- 功能描述：从立意、押韵、对仗、意境、语言五个维度进行评分。
- 技术实现：通过 `/api/scoring` 接口调用 `score_with_volcano` 函数。示例代码：

```
1 import requests
2
3 def score_with_volcano(title, content):
4     # 这里需要替换为火山引擎 API 的实际调用代码
5     url = 'https://your-volcano-api-url'
6     data = {
7         'title': title,
```

```

8         'content': content
9     }
10    response = requests.post(url, json=data)
11    return response.json()
12
13 @app.route('/api/scoring', methods=['POST'])
14 def scoring():
15     data = request.get_json()
16     title = data.get('title')
17     content = data.get('content')
18     result = score_with_volcano(title, content)
19     return result
20
21 if __name__ == '__main__':
22     app.run()

```

2.4 诗词推荐



图 2.7: 诗词推荐界面

- 功能描述：根据用户提交的诗词内容，推荐相似风格的古诗词。
- 技术实现：通过 `/api/recommend` 接口调用 `recommend_with_volcano` 函数。示例代码：

```

1 import requests
2
3 def recommend_with_volcano(title, content):
4     # 这里需要替换为火山引擎 API 的实际调用代码
5     url = 'https://your-volcano-api-url'
6     data = {
7         'title': title,

```

```

8         'content': content
9     }
10    response = requests.post(url, json=data)
11    return response.json()
12
13 @app.route('/api/recommend', methods=['POST'])
14 def recommend():
15     data = request.get_json()
16     title = data.get('title')
17     content = data.get('content')
18     result = recommend_with_volcano(title, content)
19     return result
20
21 if __name__ == '__main__':
22     app.run()

```

3 文言文分析模块

3.1 逐句翻译



图 2.8: 诗词翻译界面

- 功能描述：将文言文逐句翻译为现代汉语。
- 技术实现：通过 `/api/translate` 接口调用火山引擎 API。示例代码：

```

1  import requests
2
3  @app.route('/api/translate', methods=['POST'])
4  def translate():
5      content = request.get_json().get('content')
6      url = 'https://your-volcano-api-url'
7      data = {
8          'content': content
9      }
10     response = requests.post(url, json=data)
11     return response.json()
12
13 if __name__ == '__main__':
14     app.run()

```

3.2 重点字词解释

3.3 重点字词解释

逐句翻译

重点字词

主旨分析

通假字

有通又

“有”是“又”的通假字，用于连接整数与零数，表示二十一年后还有余数。

古今异义词

卑鄙

古义：身份低微，见识短浅
今义：（语言、行为）恶劣；不道德
“卑鄙”在古代为中性词，“卑”指地位低下，“鄙”指见识浅陋；现代为贬义词，侧重品德低劣，含义和感情色彩均发生变化。

感激

古义：感动而奋发
今义：因对方好意或帮助而感动并产生

逐句翻译

重点字词

主旨分析

词类活用

躬

原词性：名词（身体）
活用为：状语（副词性）
“躬”本为名词，指“身体”；在“躬耕于南阳”中活用为状语，表示“亲自”，修饰动词“耕”，功能为说明动作的方式。

特殊句式

状语后置（介词结构后置）

例句：躬耕于南阳
正常语序应为“于南阳躬耕”，介词结构“于南阳”作状语后置，强调耕作的地点。

状语后置（介词结构后置）

例句：苟全性命于乱世
正常语序应为“于乱世苟全性命”，介词结构“于乱世”作状语后置，强调保全性命的背景。

- 功能描述：分析通假字、古今异义词、词类活用和特殊句式。
- 技术实现：通过 `/api/explain-words` 接口进行语言现象分析。示例代码：

```
1 @app.route('/api/explain-words', methods=['POST'])
2 def explain_words():
3     content = request.get_json().get('content')
4     # 这里需要实现具体的语言现象分析逻辑
5     result = {'success': True, 'result': '字词解释结果'}
6     return result
7
8 if __name__ == '__main__':
9     app.run()
```

3.4 主旨分析



图 2.9: 文言文主旨分析界面

- 功能描述：从中心思想、艺术特色、历史背景三方面分析。
- 技术实现：通过 `/api/analyze-theme` 接口进行主旨分析。示例代码：

```
1 @app.route('/api/analyze-theme', methods=['POST'])
2 def analyze_theme():
```

```

3     content = request.get_json().get('content')
4     # 这里需要实现具体的主旨分析逻辑
5     analysis = {'success': True, 'analysis': '主旨分析结果'}
6     return analysis
7
8 if __name__ == '__main__':
9     app.run()

```

4 素材库模块



图 2.10: 素材库界面 1

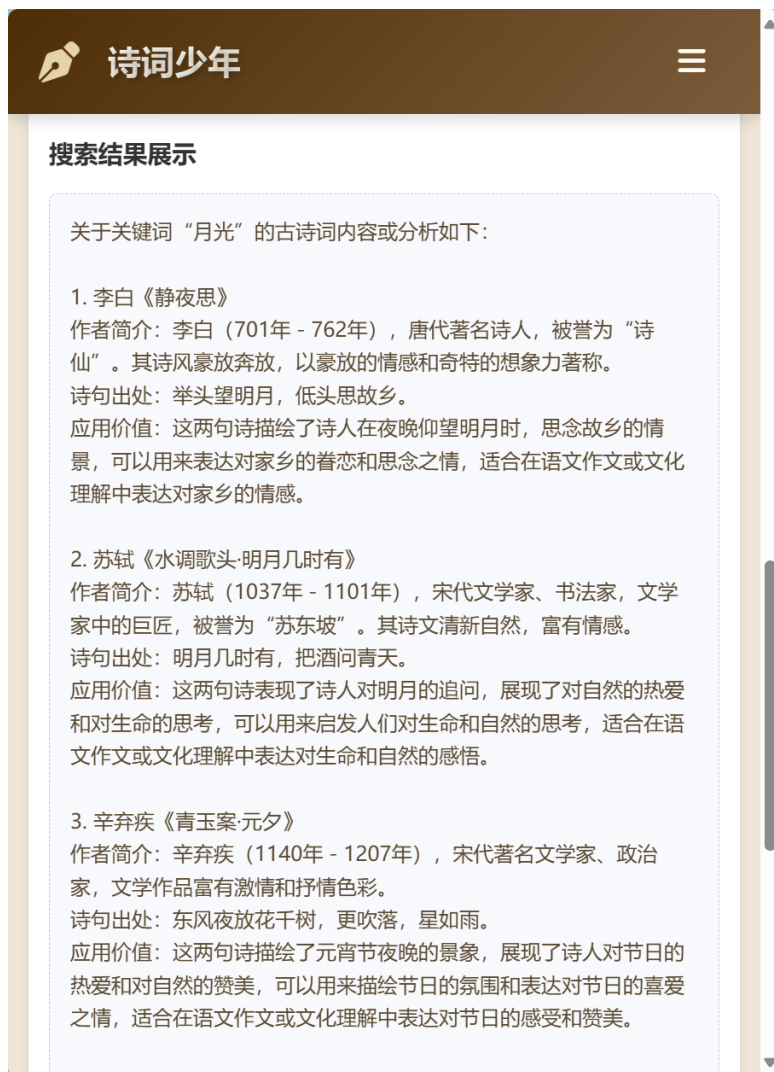


图 2.11: 素材库界面 2

- 功能描述：根据关键词提供相关古诗词内容及分析说明。
- 技术实现：通过 /search 接口调用 ChatAnywhere API。示例代码：

```

1  import requests
2
3  @app.route('/search', methods=['POST'])
4  def search():
5      query = request.get_json().get('query')
6      url = 'https://your-chatanywhere-api-url'
7      data = {
8          'query': query
9      }
10     response = requests.post(url, json=data)
11     return response.json()

```

```
12
13 if __name__ == '__main__':
14     app.run()
```

5 图文生成模块



图 2.12: 图文生成界面 1



图 2.13: 图文生成界面 2

- 功能描述：根据上传图片生成七言律诗。
- 技术实现：通过 `/generate_poem` 接口调用火山引擎 API。示例代码：

```

1  import requests
2
3  @app.route('/generate_poem', methods=['POST'])
4  def generate_poem():
5      image = request.files.get('image')
6      url = 'https://your-volcano-api-url'
7      files = {'image': image}
8      response = requests.post(url, files=files)
9      return response.json()
10

```

```
11 if __name__ == '__main__':  
12     app.run()
```

项目架构

1 前端架构

1.1 技术选型

- 基础框架：采用 HTML5 + CSS3 + JavaScript 原生技术栈，确保跨平台兼容性
- UI 组件库：集成 Element UI 框架，提供丰富的交互组件和统一的视觉风格
- 状态管理：使用 LocalStorage 存储用户会话信息和偏好设置
- 异步通信：基于 Fetch API 实现前后端数据交互，支持 Promise 链式调用

1.2 模块化设计

前端代码按照功能模块组织，主要目录结构：

- /templates：存放各功能页面的 HTML 文件
- /assets：静态资源目录，包含图片、字体等
- /css：CSS 样式文件，采用模块化设计
- /js：JavaScript 逻辑代码，按功能拆分

1.3 核心交互流程

1. 用户操作触发前端事件（如点击按钮、提交表单）
2. JavaScript 捕获事件并进行数据预处理
3. 通过 Fetch API 发送请求到后端 API 接口
4. 接收后端响应数据并更新 UI 显示
5. 使用动画效果提升用户体验（如加载中状态、过渡效果）

2 后端架构

2.1 技术栈

- **Web 框架**: Flask 微框架, 轻量级且易于扩展
- **API 设计**: 遵循 RESTful 架构风格, 统一接口规范
- **数据库**: SQLite 作为嵌入式数据库, 适合中小规模应用
- **缓存机制**: 使用 Flask-Caching 实现 API 响应缓存

2.2 模块化设计

后端代码采用蓝图 (Blueprint) 机制组织, 主要模块:

- **auth**: 用户认证与授权模块
- **poem**: 诗词创作相关功能
- **classical**: 文言文分析模块
- **material**: 素材库管理模块
- **paint**: 图文生成模块

2.3 API 网关设计

- 统一请求入口, 处理跨域、限流、认证等通用逻辑
- 基于 Flask 的中间件机制实现请求拦截和预处理
- 提供 API 文档和测试界面
- 实现请求参数校验和数据格式转换

3 系统部署架构

3.1 开发环境

- 前端: VS Code + Live Server 插件
- 后端: PyCharm + Flask 内置开发服务器

- 数据库：SQLite 命令行工具
- 版本控制：Git + GitHub 协作开发

3.2 第三方 API 调用

- 问题：火山引擎和 ChatAnywhere API 的调用限制和稳定性
- 解决方案：
 - 实现 API 调用限流和熔断机制
 - 设计本地缓存策略，减少外部依赖
 - 开发降级方案，在第三方服务异常时提供基础功能

数据库操作技术

1 Python 与 SQLite 交互

- 连接数据库：sqlite3.connect()。示例代码：

```
1 import sqlite3
2 conn = sqlite3.connect('users.db')
```

- 执行 SQL：execute()。示例代码：

```
1 cursor = conn.cursor()
2 cursor.execute('CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY AUTOINCREMENT, ...
                username TEXT, password TEXT)')
```

- 事务处理：commit() 和 rollback()。示例代码：

```
1 try:
2     cursor.execute('INSERT INTO users (username, password) VALUES (?,?)', ('testuser', ...
                                'testpassword'))
3     conn.commit()
4 except:
5     conn.rollback()
```

- 参数化查询：防止 SQL 注入。示例代码：

```
1 username = 'testuser'
2 cursor.execute('SELECT * FROM users WHERE username =?', (username,))
```

2 数据库安全措施

- 密码加密：generate_password_hash 和 check_password_hash。在用户注册和登录模块中使用这两个函数来加密和验证密码。

- 输入验证：防止非法数据。在用户输入数据时，需要对输入进行验证，确保数据的合法性。
- 参数化查询：避免 SQL 注入。在执行 SQL 查询时，使用参数化查询可以有效防止 SQL 注入攻击。

3 数据库备份与恢复

- 定期备份数据库文件：可以使用脚本定期复制 `users.db` 文件到备份目录。
- 实现数据恢复功能：在需要恢复数据时，将备份的数据库文件复制回原位置。

API 接口说明

1 API 设计规范

1.1 URL 规范

- 使用名词复数表示资源集合，如 /api/poems
- 使用 HTTP 动词表示操作，如 GET/POST/PUT/DELETE
- 采用小写字母，使用连字符 (-) 分隔单词
- 版本控制：在 URL 中包含版本号，如 /v1/api/poems

1.2 响应格式

统一采用 JSON 格式返回数据，标准响应结构：

```

1 {
2   "code": 200,           // 状态码
3   "message": "操作成功", // 消息描述
4   "data": {             // 数据内容
5     // 具体数据结构
6   },
7   "timestamp": 1634567890 // 时间戳
8 }
```

2 用户认证模块

2.1 用户登录

- 请求参数：
 - username (string, required): 用户名
 - password (string, required): 密码
- 响应数据：

- token (string): 认证令牌
- user (object): 用户信息
- 示例:

```

1  {
2    "code": 200,
3    "message": "登录成功",
4    "data": {
5      "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
6      "user": {
7        "id": 1,
8        "username": "testuser",
9        "email": "test@example.com"
10     }
11   }
12 }
```

2.2 用户注册

- 请求参数:
 - username (string, required): 用户名
 - password (string, required): 密码
 - email (string, required): 邮箱
- 响应数据:
 - user (object): 用户信息

3 诗词创作模块

3.1 诗词提交

- 请求参数:
 - title (string, required): 标题
 - content (string, required): 内容
 - style (string): 风格

- theme (string): 主题
- 响应数据:
 - poem (object): 诗词信息

3.2 诗词润色

- 请求参数:
 - content (string, required): 诗词内容
 - style (string): 润色风格 (elegant, classical)
- 响应数据:
 - polishedContent (string): 润色后的内容

3.3 诗词评分

- 请求参数:
 - content (string, required): 诗词内容
- 响应数据:
 - score (float): 总分
 - details (object): 各维度评分详情

4 文言文分析模块

4.1 文言文翻译界面

- 请求参数:
 - content (string, required): 文言文内容
- 响应数据:
 - translation (string): 翻译结果

4.2 重点字词解释

- 请求参数：
 - content (string, required): 文言文内容
- 响应数据：
 - words (array): 字词解释列表

5 素材库模块

5.1 获取素材列表

- 请求参数：
 - keyword (string): 搜索关键词
 - page (integer): 页码
 - pageSize (integer): 每页数量
- 响应数据：
 - total (integer): 总数
 - items (array): 素材列表

5.2 获取单个素材详情

- 响应数据：
 - material (object): 素材详情

6 图文生成模块

6.1 根据图片生成诗词

- 请求参数：
 - image (file, required): 图片文件
- 响应数据：

- poem (object): 生成的诗词

部署与运行

1 环境准备

- Python 3
- Flask
- OpenAI Python 库
- SQLite

2 配置文件

- 在各模块配置火山引擎和 ChatAnywhere 的 API 密钥。需要在相应的 Python 文件中设置 API 密钥，例如：

```
1 VOLCANO_API_KEY = 'your_volcano_api_key'
2 CHATANYWHERE_API_KEY = 'your_chatanywhere_api_key'
```

3 运行步骤

1. 安装依赖： `pip install -r requirements.txt`
2. 创建数据库：创建 `users.db` 和 `users` 表。可以使用以下 SQL 语句创建表：

```
1 CREATE TABLE IF NOT EXISTS users (
2     id INTEGER PRIMARY KEY AUTOINCREMENT,
3     username TEXT,
4     password TEXT
5 );
```

3. 启动应用： `python app.py`
4. 访问应用： `http://127.0.0.1:5000`

差异化创新

1 产品定位创新

1.1 多场景覆盖

“诗文少年”是市场上首个集诗词创作、文言文分析、素材检索和图文生成于一体的综合性平台，满足了用户从学习到创作的全流程需求。

1.2 精准用户画像

聚焦中学生群体，针对其学习痛点设计功能，例如：

- 中考高考文言文知识点解析
- 作文素材智能推荐
- 符合中学生认知水平的诗词评分体系

2 技术应用创新

2.1 AI 多模态融合

平台创新性地将文本处理与图像识别技术结合：

- 基于图像生成诗词：通过分析图片内容生成相关主题的七言律诗
- 图文互译：支持将诗词描述转换为图像，或从图像中提取诗词灵感

2.2 自适应学习系统

根据用户的使用历史和创作水平，提供个性化的学习路径：

- 初级用户：提供更多基础素材和创作指导
- 高级用户：推荐更具挑战性的创作任务和风格探索

3 用户体验创新

3.1 低门槛创作

通过 AI 辅助降低诗词创作难度：

- 智能补全：在用户创作过程中提供字词建议
- 押韵检测：实时检测诗词押韵情况并提供修改建议
- 格律检查：自动识别并提示不符合格律的部分

3.2 沉浸式学习环境

设计符合传统文化审美的界面：

- 采用中国风配色方案，使用宣纸纹理等元素
- 动态水墨动画效果，模拟传统书写体验
- 背景音乐选择：提供古琴、古筝等传统音乐作为创作背景